

Function Calling in the APARA C Compiler

How C Function Calls Are Lowered to APARA mcode

APARA C Compiler Project

June 22, 2026

1 Introduction and Strategy

A C function call touches more compiler machinery at once than almost any other construct: the IR generator (one node, `IRCall`), the register allocator (every live value must survive a call whose callee may clobber anything), and the code generator's calling-convention logic (arguments in, return value out, the stack frame itself). This report covers all three, end to end, for direct calls; indirect calls through a function pointer are mentioned only briefly, since they are blocked at the assembler level for an unrelated reason (Section 5).

The governing strategy has three deliberate simplifications, each trading some performance for a much smaller, more predictable amount of code to get right:

1. **Fixed argument registers, no stack-passed arguments.** Exactly four registers, `$r2–$r5`, carry the first four arguments; a 5th argument is a hard compile-time error, not a silent truncation or a fallback to stack passing.
2. **Full caller-save, not a fixed callee-save/caller-save split.** Every register the allocator considers *currently live* at a call site — not a fixed subset chosen by an ABI table — is spilled to the stack before the call and reloaded after. The callee is therefore free to clobber every register except `$r26/$r27` (FP/SP, never touched by the allocator at all).
3. **One epilogue per function, no matter how many return statements.** Every `return` just loads the result into `$r1` and jumps to a single, shared epilogue label; the actual stack teardown and `$return` instruction exist exactly once per function.

2 The Call Site (Caller Side): Algorithm

`ir_gen.py`'s `_call` lowers an ordinary call to exactly one IR node, `IRCall(dest, func_name, args)` (`ir_gen.py:1188`) — all of the complexity below is `codegen.py`'s `_gen_IRCall` (`codegen.py:943–1018`) turning that one node into real mcode.

Listing 1: `_gen_IRCall`, `codegen.py:943–1018` (essence)

```

1 function EMIT-CALL(dest, func_name, args):
2     if len(args) > 4: COMPILE-ERROR("too many arguments"); exit
3
4     // 1. Caller-save: spill every currently-live register to the stack
5     saved <- current allocator map (name -> register), as a list
6     for (name, reg) in saved:
7         slot <- next caller-save stack slot
8         emit "$st [FP + slot], reg"
9         remember slot for name
10
11     // 2. Set up arguments in $r2..$r5, reading from the JUST-SAVED stack
12     //     slots (not from registers) so writing arg[i] can never clobber
13     //     a register that arg[i+1] still needs to read

```

```

14  for i, arg in enumerate(args[:4]):
15      if arg is a live temp that was just saved:  emit "$ld ARG[i], [FP + slot]"
16      elif arg is a constant:                    load it into ARG[i] directly
17      else:                                       emit "+ ARG[i] = 0 + arg's
register"

18
19  // 3. Make the call
20  emit "$call func_name"
21  // From here on every register except $r1 may hold garbage --
22  // the allocator's bookkeeping is now stale; only the caller-save
23  // slots on the stack are trustworthy.
24
25  // 4. Capture the return value
26  if dest is needed:
27      if no free register exists:
28          EVICT-ONE-CALLER-MAVED-VALUE-TO-A-SPIII-SLOT()  // see Section 2.1
29      reg <- allocate a register for dest
30      emit "+ reg = 0 + $r1"
31
32  // 5. Restore every caller-saved value from its stack slot
33  for (name, reg, slot) in saved:
34      emit "$ld reg, [FP + slot]"

```

The one tricky edge case: capturing the return value when the pool is full

If all 28 registers were already live before the call, step 4 has nowhere to put the return value — every register is still recorded as holding some caller-saved value (even though, physically, the call just clobbered it). The fix: pick any one saved value that is *not* currently sitting in $\$r1$, reload its correct value from its caller-save slot, write it straight back out to a dedicated long-term spill slot instead, and mark its register free. This never touches $\$r1$ itself, so the live return value is never at risk of being overwritten while making room for it.

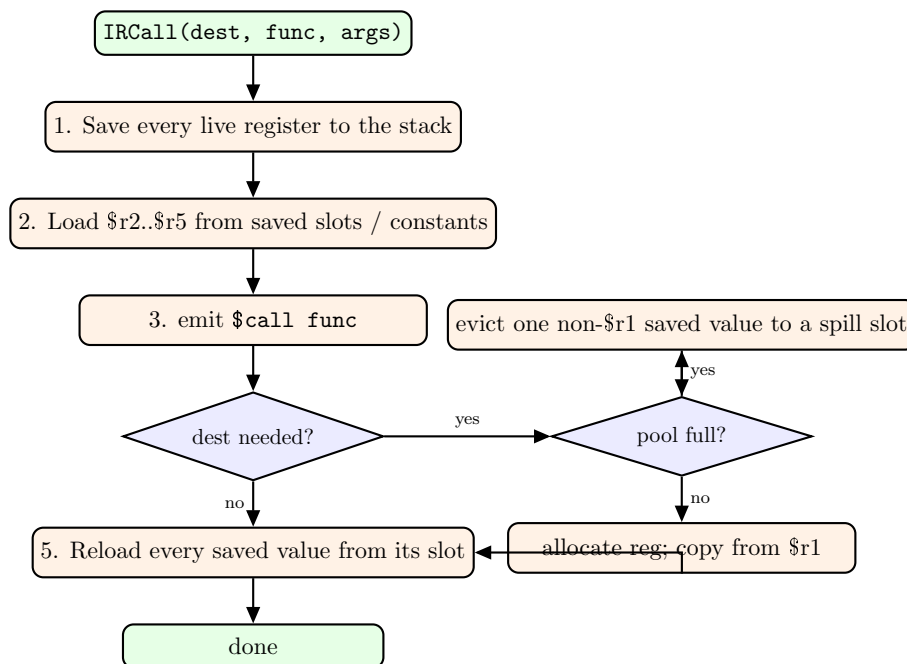


Figure 1: Call-site code generation, step by step. The eviction loop (top right) only ever runs once per call, since one evicted register is always enough to make room for the single return-value destination.

3 Function Entry and Exit (Callee Side): Algorithm

Listing 2: `_gen_IRFuncBegin / _gen_IRFuncEnd`, `codegen.py:572–619` (essence)

```

1 function EMIT-PROLOGUE(func_name, params, frame_size):
2     if len(params) > 4: COMPILE-ERROR("too many parameters"); exit
3     reset register allocator to a fresh 28-register pool    // new function, clean
4                                                             // slate
5     LABEL func_name
6     emit "$st [SP + 0], FP"                                // save caller's frame pointer
7     emit "+ FP = 0 + SP"                                    // establish the new frame
8     total <- frame_size + CALLER_SAVE_BYTES + SPILL_RESERVE
9     emit "- SP = SP - total"                                // carve out locals + caller-save + spill
10                                                             // areas
11     for i, (name, fp_offset, width) in enumerate(params):
12         emit "$st (width) [FP + fp_offset], ARG[i]"        // width-correct, not a blind
13                                                             // $i64
14 function EMIT-EPILOGUE(func_name):
15     LABEL func_name + "_epilogue"                          // every "return" jumps here, never falls
16                                                             // through
17     emit "+ SP = 0 + FP"                                    // collapse the frame
18     emit "$ld FP, [SP + 0]"                                // restore caller's frame pointer
19     emit "$return"
```

The parameter-store width matters: every parameter is written back at its *real* C width (\$i32 for a plain int, \$i8 for a char, ...), not a blanket 64-bit store. APARA's narrow load always reads a fixed bit-slice of the 8-byte DMEM word; storing a small value as a full 64-bit quantity and then reading it back narrow would silently read the wrong half. (This was a real, confirmed bug in this compiler before it was fixed — see Section 5.)

Listing 3: `_gen_IRReturn`, `codegen.py:1104–1111`

```

1 function EMIT-RETURN(value):
2     if value exists:
3         emit "+ $r1 = 0 + value's register"                // load the result into RET
4         emit unconditional jump to func_name + "_epilogue"
```

A function with three `return` statements compiles to three small load-into-\$r1-then-jump sequences, all converging on the *same* epilogue — the stack-teardown code is never duplicated.

4 A Fully Compiled, Verified Worked Example

Listing 4: Source

```

1 int add(int a, int b) { return a + b; }
2 int main() {
3     int r;
4     r = add(3, 4);
5     return r;
6 }
```

Listing 5: Generated mcode (both functions, in full)

```

1 add:
2 ||
```

```

3      $st ($i64) [$r27 + 0] $r26          // prologue: save caller's FP
4      ;
5      ||
6      + $r26 ($i64) $r0 $r27             // FP = SP
7      $set $r1 0 824                     // 824 = frame(88) + caller-save(224) + spill
      (512)
8      ;
9      ||
10     - $r27 ($i64) $r27 $r1             // SP -= 824
11     $st ($i32) [$r26 + -8] $r2         // store param a (width = $i32, matches "int")
12     $st ($i32) [$r26 + -16] $r3        // store param b
13     + $r1 ($i64) $r26 -8              // body starts: &a
14     ;
15     ||
16     $ld ($i32) $r2 [$r1 + 0]           // load a
17     + $r3 ($i64) $r26 -16
18     ;
19     ||
20     $ld ($i32) $r4 [$r3 + 0]           // load b
21     ;
22     ||
23     + $r5 ($i64) $r2 $r4              // r5 = a + b
24     ;
25     ||
26     + $r1 ($i64) $r0 $r5              // RET = r5
27     ? ($i64) $r0 == $goto add_epilogue
28     ;
29 add_epilogue:
30     ||
31     + $r27 ($i64) $r0 $r26            // SP = FP
32     ;
33     ||
34     $ld ($i64) $r26 [$r27 + 0]         // FP = caller's FP
35     $return
36     ;
37 main:
38     ||
39     $st ($i64) [$r27 + 0] $r26
40     ;
41     ||
42     + $r26 ($i64) $r0 $r27
43     $set $r1 0 816
44     ;
45     ||
46     - $r27 ($i64) $r27 $r1             // nothing live yet -- caller-save loop is empty
47     + $r2 ($i64) $r0 3                 // arg0 = 3
48     + $r3 ($i64) $r0 4                 // arg1 = 4
49     ;
50     ||
51     $call add
52     ;
53     ||
54     + $r1 ($i64) $r0 $r1               // capture RET (dest happened to land on $r1 too
    )
55     + $r2 ($i64) $r26 -8              // &r, for the store below
56     ;
57     ||
58     $st ($i32) [$r2 + 0] $r1          // r = (the call's result)

```

```

59     + $r3 ($i64) $r26 -8
60 ;
61 ||
62     $ld ($i32) $r4 [$r3 + 0]
63 ;
64 ||
65     + $r1 ($i64) $r0 $r4
66     ? ($i64) $r0 == $goto main_epilogue
67 ;
68 main_epilogue:
69 ||
70     + $r27 ($i64) $r0 $r26
71 ;
72 ||
73     $ld ($i64) $r26 [$r27 + 0]
74     $return
75 ;

```

Note the line `+ $r1 = $r0 + $r1` in `main`: this looks like a no-op, and on these exact values it is one — but it is not special-cased anywhere. `main` had nothing live at the call site, so the caller-save loop did nothing, and the register allocator’s ordinary linear-scan happened to hand out `$r1` (the first register in its free list) for the call’s destination temp, which collides with `RET` (`= $r1`) itself. The algorithm is correct either way — it would be a real copy into a *different* register if anything else had been live first.

Caller-save actually firing: a second example

`r = x + add(3, 4);` with `x` already assigned beforehand produces, at the call site:

Listing 6: Caller-save in action (excerpt)

```

1     $st ($i64) [$r26 + -96] $r3      // save x's value before clobbering $r3 for an
    argument
2     + $r2 ($i64) $r0 3
3 ;
4 ||
5     + $r3 ($i64) $r0 4              // $r3 reused for arg1 -- safe, x is already on
    the stack
6     $call add
7 ;
8 ||
9     + $r4 ($i64) $r0 $r1            // capture return value into a fresh register
10    $ld ($i64) $r3 [$r26 + -96]      // restore x
11 ;
12 ||
13    + $r5 ($i64) $r3 $r4            // r5 = x + add(3,4)

```

5 Known Limits and Historical Failure Modes

Four real, previously-confirmed defects sit directly in this machinery and are worth recording alongside the algorithm itself, since each one demonstrates a way this exact code path can go wrong if changed carelessly:

- **Narrow-parameter bug (compiler-side, fixed).** The prologue’s parameter store used to be a hardcoded `$st ($i64)` regardless of the parameter’s real width, while reads correctly used the narrow width — so any `int/short/char` parameter silently read back

as garbage. Fixed by threading the real element width through `IRFuncBegin.params` (Section 3's width-correct store is the fix).

- **4-argument ceiling (compiler-side, fixed).** A 5th argument or parameter used to be silently dropped with no error. Now both `_gen_IRCall` and `_gen_IRFuncBegin` fail loudly at compile time instead.
- **\$call disassembler sign-extend bug (simulator-side, fixed).** Every *backward* call (a callee defined above its caller in source order — the ordinary C pattern) resolved to the wrong target entirely, because the disassembler sign-extended the wrong bit of the 25-bit jump field. Fixed in the simulator; confirmed present in the current authoritative build.
- **Instruction-memory size bug (simulator-side, fixed).** Functions whose compiled body exceeded 2048 words were silently truncated, which surfaced as *this* machinery's symptoms (wrong return values) despite the bug having nothing to do with calling convention logic at all.

Indirect calls through a function pointer (`$call $rN`) are valid hardware behaviour and fully implemented on the compiler side (`IRFuncAddr/IRIndirectCall`), but are blocked end-to-end because no instruction exists to load a function's absolute address into a register in the first place — a hard assembler limitation, not a bug in this calling convention.

6 Summary

A C function call is, by the time it reaches mcode, five mechanical steps: spill whatever is live, load the fixed argument registers from the safety of the stack rather than from registers that are about to be overwritten, call, capture `$r1`, and reload everything that was spilled. A function definition is two mechanical bookends around an ordinary instruction stream: a prologue that links a new frame onto the old one and stores incoming arguments at their real width, and a single epilogue, shared by every `return` in the function, that unwinds the frame and hands control back. Nothing here depends on what kind of function is being called or how many times it is called from — the same five-step/two-bookend pattern produced every line in Section 4's worked example, and produces every function call in every program this compiler has ever built.